01 PyHelp AI

Assistant de support Python fondé sur RAG
Rapport technique - Projet 1 MLOps / LLMOps
ENSAF · ISDIA3 · Année universitaire 2026-2027
Livraison du 8 novembre 2026

Problème et utilisateurs

Les étudiants et développeurs débutants rencontrent des difficultés récurrentes sur la syntaxe, les structures de données, les imports, les exceptions et les environnements virtuels. Une réponse plausible mais fausse peut installer une mauvaise compréhension ou conduire à des manipulations inadaptées. PyHelp AI recherche des extraits dans un corpus de documentation Python, puis les transmet à un modèle pour produire une réponse sourcée ou une abstention.

Le périmètre est volontairement réduit à neuf fichiers anglais de la documentation fournie, présentée comme Python 3.11.18. Les questions peuvent être formulées en français ou en anglais. Le système ne garantit pas les réponses sur les autres versions, ne lance aucun code et ne modifie aucun environnement utilisateur. Les cas sans preuve suffisante doivent conduire à un refus ou à une demande de précision.

Résultat de l’étude

La chaîne documentaire, l’index FAISS, l’API et les contrôles de reproductibilité ont été exécutés dans l’environnement Linux disponible. La configuration B est retenue sur le jeu de développement. Les mesures de retrieval montrent un bénéfice du découpage 800/120 avec top-k=5, tout en révélant une qualité inférieure en français. Une campagne distincte évalue un véritable petit LLM local. Les mocks des tests ne sont jamais utilisés pour annoncer les métriques de qualité.

La livraison constitue un démonstrateur académique instrumenté. Elle rend visibles les erreurs au lieu de les masquer : retrouver un passage ne signifie pas que le modèle le comprendra, et une citation valide ne prouve pas qu’une affirmation est vraie. La promotion vers un usage public doit rester conditionnée à une évaluation de génération suffisamment satisfaisante et à une revue de sécurité.

Livrables et lecture

Le dépôt comprend le corpus préparé, les manifestes, deux configurations expérimentales, le jeu annoté, les runs MLflow, les tests pytest, le workflow GitHub Actions, les scripts PowerShell, le monitoring et un scénario de démonstration de deux minutes. Les preuves détaillées sont conservées sous reports/. La matrice reports/conformite.md relie les exigences du PDF aux composants et aux contrôles.

02 Exigences et décisions

L’analyse porte sur le cahier des charges complet, notamment les sections 2, 3, 7, 8, 9, 10 et 11. Le projet 1 exige un RAG monolithique et un index vectoriel unique. Les extensions multi-index, les agents et le fine-tuning sont exclus de cette réalisation. L’effort principal porte sur les données, les expériences, les contrôles et les preuves de reconstruction.

Exigence	Réalisation	Vérification
2 : socle commun	Git, DVC, MLflow, pytest, FastAPI	Commit local, dvc.lock, runs et tests
3 : RAG complet	Ingestion, embeddings, FAISS, contexte, LLM	Pipeline exécuté ; API et sources contrôlées
3.5 : évaluation	42 questions, preuves exactes, dev/test	Résultats détaillés et analyses FR/EN
7 : extensions facultatives	Aucune architecture complexe	Un seul index actif, pas de routing
8 : quality gates	Intégrité, tests et recherche rejouée	Échec si perte face à la baseline figée
9 : monitoring	JSONL, latence, erreurs, chunks, RSS	Simulation HTTP et rapport agrégé
10 : livrables	Rapport, reproductibilité, démonstration	Dossier et scénario minuté
11 : cadrage et risques	Droits, versions, limites et gouvernance	Inventaire et registre des risques

Choix techniques motivés

Un assemblage Python direct est retenu : LangChain aurait ajouté une couche sans simplifier ce petit pipeline. FAISS IndexFlatIP réalise une recherche exacte sur des vecteurs normalisés ; le produit scalaire correspond alors à la similarité cosinus. Ce choix évite de régler un index approximatif sur quelques centaines de chunks. Les données restent locales par défaut.

Sentence Transformers fournit un encodeur multilingue MiniLM à 384 dimensions. La révision du modèle est figée. Ollama est le mode de service proposé sur le poste étudiant. Les options Mistral et Gemini existent mais sont désactivées sans opt-in explicite ; aucun service payant n’a été appelé. Un connecteur Transformers CPU est ajouté pour permettre une véritable expérience locale dans cet environnement sans Ollama.

La préférence initiale Python 3.10 a été réexaminée à la suite de l’audit : cette version a atteint sa fin de support le 1er octobre 2026 et les dépendances récentes corrigées demandent Python 3.11+. Le projet cible donc Python 3.11/3.12 ; les commandes Windows sont documentées, mais les exécutions de cette livraison ont eu lieu sous Linux/Python 3.12. Le contenu documentaire demeure 3.11.18.

03 Corpus, droits et provenance

L’archive fournie contient 513 entrées, dont 499 fichiers TXT. Le contrôle CRC ne signale aucune erreur et les 499 fichiers se décodent en UTF-8. Seuls les neuf fichiers approuvés sont extraits comme documents de travail : 235,113 octets au total et 181 sections repérées. Aucun autre chapitre n’a été ajouté après l’évaluation ; les lacunes constatées ne justifient pas une modification silencieuse du périmètre.

Source sélectionnée	Octets	Sections
tutorial/introduction.txt	18061	6
tutorial/controlflow.txt	37796	23
tutorial/datastructures.txt	23770	13
tutorial/modules.txt	24573	11
tutorial/errors.txt	22064	11
tutorial/venv.txt	7121	4
faq/programming.txt	74605	75
faq/general.txt	18964	26
installing/index.txt	8159	12

Conservation et traçabilité

data/source/ contient l’archive originale ; data/raw/ contient les octets intacts des neuf fichiers. Les versions nettoyées sont séparées dans data/processed/documents/. L’inventaire associe chemin, titre, version, volume, sections et empreintes SHA256 avant et après nettoyage. L’empreinte de l’archive commence par e3cf166d4e090f36 et figure entièrement dans configs/corpus.json. La sélection est explicite et sa modification exige une revue.

Le nom et les textes internes identifient la documentation Python 3.11.18. En revanche, l’URL exacte du téléchargement initial et une signature de l’éditeur ne sont pas fournies : l’origine directe est donc notée « archive fournie par l’utilisateur ». Cette nuance évite de confondre une structure cohérente avec une attestation cryptographique officielle.

Droits et anomalies de contenu

Les textes embarqués de licence et de copyright sont conservés sous legal/, hors index. La documentation relève de la licence PSF ; les exemples de code relèvent des dispositions PSF/Zero-Clause BSD indiquées dans ces textes. Les mentions ne sont pas supprimées. La FAQ contient des formulations historiques : une version documentaire récente n’implique pas que chaque paragraphe présente une situation actuelle. Les fixtures contradictoires ou malveillantes sont isolées dans les tests et ne modifient jamais ce corpus.

04 Ingestion et architecture

Sécurité de l’extraction

Le pipeline ne fait pas appel à extractall. Il valide les chemins POSIX, refuse les traversées avec « .. », les chemins absolus, les séparateurs Windows suspects, les liens symboliques et les noms en double. Les volumes sont bornés avant lecture et le CRC est vérifié. La sélection doit correspondre exactement aux chemins configurés. Le décodage UTF-8 est strict : une erreur interrompt la préparation au lieu de remplacer silencieusement des caractères.

Nettoyage et découpage

Le nettoyage normalise seulement les fins de ligne et retire les caractères de saut de page. Il ne compresse pas les espaces et ne réindente pas les exemples. Les titres sont repérés à partir des soulignements de la documentation texte. Les chunks restent dans une section et privilégient une frontière de ligne lorsque cela est possible. Un bloc de code long peut toutefois être réparti sur plusieurs chunks : le chevauchement réduit la rupture, sans garantir un programme complet dans chaque fragment.

Chaque chunk conserve un identifiant stable dérivé de l’empreinte du document, des positions et du texte ; il conserve aussi document_id, source_path, titre, section, version, positions et contenu. Les positions portent sur le document nettoyé, les empreintes des octets bruts assurant la liaison avec l’original. Deux préparations identiques produisent les mêmes identifiants.

Étape	Entrée / sortie	Contrôle
Préparer	ZIP → raw + processed + inventaire	Sélection, UTF-8, CRC, SHA256
Indexer	Chunks → embeddings → FAISS	Révision modèle, dimension, empreinte
Rechercher	Question → top-k commun	Scores et provenance réelle
Générer	Question + extraits → JSON	Abstention et références autorisées
Servir	FastAPI → interface HTML	Validation, 422/503 et télémétrie

Un seul index en service

Les deux index A et B sont des artefacts de comparaison hors ligne. Le service ne route pas entre eux : il charge uniquement artifacts/current/index.faiss. À l’initialisation, l’empreinte de l’index et celle des chunks sont vérifiées avant lecture. L’encodeur de requête doit être celui de construction ; toute modification demande une reconstruction et une nouvelle campagne.

Le modèle MiniLM conserve sa limite de tokens native, plus petite que certains fragments de 800 caractères. Une troncature d’encodage peut donc contribuer aux échecs : les caractères ne sont pas des tokens. Cette limite est documentée et n’est pas masquée par une augmentation artificielle du paramètre de longueur.

05 Expériences de retrieval

Le corpus, l’encodeur et ses poids sont identiques dans les deux expériences. Les index sont reconstruits à chaque configuration. Le jeu compte 32 questions répondables et 10 scénarios négatifs ; 15 questions sont réservées au développement et 27 au test. La sélection se fait sur le développement. Les annotations ont été vérifiées contre des extraits et positions réels, mais restent à faire relire par un évaluateur humain indépendant.

Mesure	Configuration A	Configuration B
Taille / overlap / top-k	400 / 80 / 3	800 / 120 / 5
Nombre de chunks	844	423
Construction, secondes	27.4	13.9
Recall@k développement	40.0 %	60.0 %
Recall@k test	50.0 %	65.9 %
Précision passages test	22.7 %	15.5 %
Recall test français	36.4 %	45.5 %
Recall test anglais	63.6 %	86.4 %
Latence retrieval test moyenne, ms	16.4	22.3
Latence retrieval test P95, ms	22.1	34.7

Définition des mesures

Recall@k désigne ici le rappel de groupes de preuves attendues. Un groupe est récupéré lorsqu’un chunk du bon document recouvre au moins 40 caractères du passage annoté, ou le passage entier s’il est plus court. Une question à plusieurs passages peut obtenir un score partiel. La précision mesure la proportion de chunks répondant à ce critère. Ce recouvrement n’est pas une validation sémantique ; les détails sont dans grille_evaluation.md.

B est retenue car son rappel de développement est supérieur. Le seuil de score est ensuite calibré exclusivement sur dev, avec une grille explicitée : valeur retenue 0.400. Cela reste une heuristique fondée sur peu de négatifs ; elle ne suffit pas à détecter toute absence d’information. Les scores initiaux à 0,30 sont conservés dans les runs A/B et ne sont pas confondus avec le seuil actif calibré.

L’anglais bénéficie de la proximité linguistique avec les documents. Les reformulations françaises peuvent récupérer un exemple voisin ou une FAQ non pertinente. B augmente le contexte disponible, mais top-k=5 ajoute aussi des passages non annotés : l’amélioration du rappel s’accompagne d’une baisse de précision. Les erreurs sont conservées par question, avec scores, sections et identifiants. Taille, chevauchement et top-k changent conjointement : cette comparaison ne sépare pas leurs contributions individuelles.

06 Génération et robustesse

Le prompt fixe le périmètre Python 3.11.18, traite les documents comme des données non fiables et exige un JSON avec statut, réponse et citations. Des labels courts S1, S2… permettent au petit modèle de citer les passages ; le serveur les traduit en véritables identifiants de chunks. Une citation absente du contexte est refusée. Un JSON invalide entraîne une abstention. Ce contrôle est structurel, pas un détecteur universel d’hallucination.

La campagne locale utilise Qwen2.5-0.5B-Instruct sur CPU. Le premier essai a été interrompu après observation de formats ou références invalides. La reprise emploie les labels courts ; les deux traces sont distinguées. Le petit modèle sert à caractériser une limite de ressources, et non à établir qu’il serait suffisant pour du support de production. Les choix Ollama, Mistral et Gemini sont disponibles, mais n’ont pas été testés avec un serveur ou un compte réel dans cet environnement.

Indicateur génération	Valeur observée
Questions exécutées	42
Taux de réponse sur questions répondables	87.5 %
Abstention sur négatifs	90.0 %
Taux d’erreur technique	0.0 %
Latence moyenne / P95, ms	17904.8 / 29820.2
Citations structurellement valides parmi réponses	100.0 %
Exactitude / fidélité indépendante	Non mesurées par un évaluateur humain
Revue IA : exactitude normalisée (32)	34.4 %
Revue IA : fidélité normalisée (28 réponses)	53.6 %

Erreur importante observée

Le petit modèle peut répondre à tort que les chaînes Python sont modifiables, alors que le corpus affirme leur immutabilité. D’autres réponses restent en anglais à une question française ou sont incomplètes. La présence d’un identifiant autorisé ne rend pas ces réponses fondées. Le fichier generation.json conserve sorties, références et preuves attendues ; la revue assistée (non indépendante, notes 0/1/2) mesure exactitude, pertinence, fidélité et langue, avec dénominateurs explicités dans assisted_review_summary.json. Sept réponses sur 32 sont jugées entièrement correctes ; ces notes restent à confirmer humainement.

Les scénarios de contradiction, injection documentaire et contexte incomplet utilisent exclusivement des extraits synthétiques isolés. Les contrôles lexicaux bloquent certains motifs évidents ; ils ne garantissent pas la sécurité contre les reformulations. Les scénarios exécutés avec le LLM révisé ont bloqué l’injection explicite mais échoué sur la contradiction et le contexte incomplet. Les tests avec mocks prouvent le contrat logiciel. Seuls les scénarios exécutés avec un vrai LLM peuvent renseigner son comportement sémantique, et les résultats correspondants doivent être lus séparément.

07 Tests, CI et promotion

La suite pytest couvre l’extraction sûre, le décodage, la préservation de l’indentation, la stabilité des chunks, les métadonnées, FAISS, les citations, les refus élémentaires, les fournisseurs, les entrées HTTP, les erreurs et les logs. Les tests CI emploient un encodeur déterministe explicitement fictif et des fournisseurs simulés, sauf le rejeu de vecteurs réels détaillé ci-dessous. Aucun appel payant ni grand téléchargement n’est requis pour ces tests.

Gates automatiques

Contrôle	Origine du critère	Limite
Neuf documents, SHA256, annotations	Contrat documentaire et données approuvées	Une source autorisée peut rester incomplète
Tests essentiels tous passants	Erreurs de code ou contrat inacceptables	Un mock ne valide pas un modèle réel
Recall test au moins égal à la baseline	Baseline réelle B : 65.9 %	Petit benchmark figé, non représentatif de tous usages
Vecteurs réels et FAISS rejoués	Même index, mêmes requêtes et empreintes	Ne couvre pas un changement de l’encodeur
Audit dépendances	Aucune vulnérabilité ignorée sans revue	Instantané daté ; périmètre CI distinct du complet
Démarrage et HTTP	/health, /ask, 422, erreur 503 contrôlée	Pas de test d’hébergement public

La baseline n’est pas réécrite par un simple lancement de gate. La campagne complète peut produire une nouvelle proposition, qui doit être examinée en PR. Sur le même corpus et les mêmes questions, aucune baisse du rappel n’est autorisée. Un changement de modèle, de prompt ou de sources impose une campagne complète, une analyse des échecs et une revue de génération avant promotion. La CI seule ne donne donc pas un feu vert métier.

Sécurité des dépendances

Un premier audit a détecté des versions vulnérables, y compris dans l’environnement NLP initial. Les résultats initiaux restent archivés. Les exigences de livraison ont été révisées et un environnement corrigé est vérifié séparément ; les résultats d’audit finaux indiquent le périmètre effectivement analysé. L’audit transitif révèle aussi un avis DiskCache sans correctif publié : son risque de pickle exige un cache DVC privé (0700 sous Linux), jamais reçu d’un tiers ; l’API ne charge pas DVC. L’audit des dépendances directes ne doit pas être présenté comme un audit exhaustif de toutes les dépendances transitives ou de l’hôte.

Le workflow GitHub Actions est fourni avec des permissions de lecture et des étapes de configuration, tests, intégrité, non-régression et audit. Il a été vérifié localement par ses commandes essentielles ; son exécution sur un runner GitHub reste à effectuer après publication dans un compte autorisé. Aucun dépôt distant ni déploiement public n’a été créé.

08 Reproductibilité et service

Versions et artefacts

Git conserve code, configurations, corpus de petite taille, manifestes, index actif et résultats de référence. DVC orchestre ingestion, comparaison, préparation des vecteurs de rejeu et vérification ; son lock enregistre les empreintes. Les sorties sont configurées cache:false car leurs volumes restent modestes et elles sont livrées dans Git. Aucun stockage DVC distant n’est nécessaire ; une migration vers un stockage externe pourra être réalisée si le corpus grandit.

MLflow conserve paramètres, métriques, révisions du modèle, empreintes de corpus et questions, prompt et artefacts d’évaluation. Les poids ne sont pas intégrés à l’archive de livraison : les références immuables et les empreintes de fichiers téléchargés permettent de les retrouver. Le premier lancement nécessite donc une connexion réseau. L’index livré permet de servir sans réindexer immédiatement.

Procédure Windows documentée

Créer un environnement Python 3.11 ou 3.12 en 64 bits, installer Torch CPU puis requirements.txt, copier .env.example vers .env et choisir un moteur. Pour Ollama, télécharger un modèle local et conserver le serveur sur localhost. Les API externes exigent PYHELP_ALLOW_EXTERNAL=1 ainsi qu’une clé et un modèle explicitement configurés. Les commandes complètes sont dans README.md et scripts/reproduce.ps1.

Commande / action	Résultat attendu
scripts.ingest puis scripts.verify	Corpus extrait et empreintes vérifiées
src.evaluation.campaign	Deux configurations suivies dans MLflow
scripts.prepare_replay puis scripts.gates	Recherche réelle rejouable en CI
python -m dvc repro	Reconstruction des étapes modifiées
uvicorn api.main:app sur 127.0.0.1:8000	API, interface HTML et OpenAPI
scripts.demo_monitoring	Scénarios HTTP et compteurs de monitoring

Contrat et gestion des erreurs

POST /ask valide une question non vide, sans caractère NUL et limitée à 2 000 caractères. Les entrées invalides retournent 422 ; une dépendance ou un fournisseur indisponible retourne 503 sans exposer le corps d’erreur distant ni une clé. Une réponse ou une abstention documentaire retourne 200, avec références, statut, raison et latence. GET /health rapporte la vie du service et la présence des artefacts : il ne prouve pas la disponibilité du LLM. Le service est destiné à localhost et ne comporte pas de contrôle d’accès pour Internet.

09 Observabilité et performances

Chaque requête reçoit un identifiant aléatoire. Les événements JSONL comprennent chemin, statut HTTP, statut applicatif, latence, nombre de chunks et mémoire RSS du processus. Les questions, réponses, identités et secrets ne sont pas écrits dans les journaux opérationnels. Les abstentions sont suivies par identifiant et compteur ; leur contenu peut être analysé séparément uniquement avec un recueil adapté et un consentement explicite.

Mesure de démonstration	Observation
Requêtes HTTP totales	5
Requêtes /ask	4
Erreurs HTTP / erreurs serveur	2 / 1
Latence /ask moyenne, ms	6263.7
Latence /ask P95, ms	21272.3
Mémoire maximale observée, Mo	1972.3
Fraction non-5xx observée	75.0 %

Interprétation des scénarios

Le script de démonstration envoie une question normale, une question hors corpus, une entrée vide et une requête déclenchant une erreur fournisseur contrôlée. Les premières requêtes passent par l’assistant réel ; seule la panne fournisseur est une fixture explicitement injectée dans le script, sans endpoint de panne en production. Le rapport distingue les erreurs de saisie 422 des erreurs serveur 503. Le 503 attendu sert à vérifier l’observabilité, pas à simuler une indisponibilité réelle prolongée.

La moyenne et le P95 sont calculés sur les observations de cette courte séance. Le premier appel peut inclure le chargement des poids ; la latence de retrieval des expériences est mesurée après un échauffement. Ces nombres ne constituent pas une mesure de charge concurrente ni un engagement de service. Le verrou de génération sérialise les appels afin de limiter la consommation mémoire ; un déploiement plus large demanderait une politique de file d’attente et de limitation de débit.

Coût, ressources et dérive

Aucun coût d’API externe n’a été engagé. Le temps CPU et la mémoire de la comparaison sont enregistrés ; la consommation électrique et le coût matériel ne sont pas mesurés. Les tokens retournés par les fournisseurs sont disponibles lorsque le moteur les expose. Une facturation externe ne peut être déduite sans tarif de compte vérifié. Pour l’exploitation locale, il faut surveiller l’évolution des erreurs, des abstentions et des latences, ainsi que les changements d’empreintes du corpus. Le taux d’abstention seul ne mesure pas la qualité des réponses.

10 Risques et suite académique

Risque prioritaire	Protection actuelle	Vérification supplémentaire
Hallucination malgré citations	Prompt, sources visibles, refus JSON invalide	Revue indépendante et meilleur LLM
Injection documentaire	Corpus hashé, contexte non fiable, motifs suspects	Tests adversariaux diversifiés
Contradictions / contexte partiel	Consigne d’abstention et fixtures isolées	Campagne sémantique plus large
Version obsolète ou différente	Mention 3.11.18 et refus de versions explicites	Comparer des sources de versions distinctes hors V1
Données personnelles / secrets	Logs minimisés, .env ignoré, localhost	Revue Git et politique de rétention
Dépendance externe	Opt-in, timeout, statut 503	Contrôle budget, contrat fournisseur
Régression ou index modifié	SHA256, DVC, Git, baseline rejouée	Revue de promotion et retour arrière

Conditions de remise et actions manuelles

Les fichiers livrés peuvent être examinés et exécutés localement. L’étudiant doit relire les annotations et les réponses, vérifier le fonctionnement sous Windows, choisir un LLM adapté aux ressources de son poste et exécuter la campagne complète avec ce modèle. Si une API est retenue, son activation et son coût doivent être autorisés avant tout appel. Il reste à publier le dépôt dans un compte GitHub autorisé, exécuter la CI distante et enregistrer la vidéo de démonstration de deux minutes.

Une publication publique ne doit pas être assimilée à une mise en production fiable. Les erreurs observées du petit modèle imposent une décision de promotion prudente : la chaîne MLOps peut fonctionner correctement alors que la génération reste insuffisante. Une amélioration peut porter sur un meilleur encodeur multilingue, une segmentation adaptée aux tokens ou un LLM plus compétent ; elle doit garder le même protocole de comparaison et ne pas transformer le test en jeu de réglage.

Références et preuves

Cahier des charges fourni : sections 2, 3, 7 à 11. Documentation Python fournie : neuf chemins dans configs/corpus.json et licences sous legal/. Références externes de mise en œuvre : docs.python.org/3.11/license.html ; sbert.net/docs/sentence_transformer/pretrained_models.html ; docs.ollama.com/api/chat ; ai.google.dev/gemini-api/docs/openai ; docs.mistral.ai/api/endpoint/chat ; devguide.python.org/versions/. Les cartes modèles sont conservées aux révisions indiquées par model_files.json.

Les preuves numériques sont séparées des appréciations : comparison.json/CSV pour le retrieval, generation.json pour les sorties réelles, robustness.json pour les fixtures, junit.xml pour les tests, monitoring.json pour les essais HTTP et dvc.lock pour le pipeline. Les valeurs non mesurées restent explicitement non mesurées. Le scénario de soutenance invite à montrer une limite ou un échec plutôt qu’à les dissimuler.